

Zadania

3.1 Pod adresem <http://127.0.0.1:3001> działa prosty serwer HTTP udostępniający dwa endpointy: <http://127.0.0.1:3001/pubkey> oraz <http://127.0.0.1:3001/privkey>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 3001:3001 --name ex1 docker.io/mazurkatarzyna/asymmetric-enc-ex1:latest
podman run -p 3001:3001 --name ex1 docker.io/mazurkatarzyna/asymmetric-enc-ex1:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 3001:3001 --name ex1 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex1:latest
podman run -p 3001:3001 --name ex1 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex1:latest
```

(b) Wygeneruj parę kluczy RSA (publiczny i prywatny). Wyeksportuj oba klucze do plików.

(c) Używając metody HTTP POST, wyślij parę kluczy RSA do serwera poprzez endpointy <http://127.0.0.1:3001/upload/public> (jako file) oraz <http://127.0.0.1:3001/upload/private> (również jako file). W odpowiedzi serwer zwróci odpowiednią informację (szczegóły klucza).

UWAGI:

- Aby wygenerować parę kluczy RSA, wykorzystaj narzędzie `OpenSSL`. Użyj argumentu [genpkey](#) oraz [pkey](#).
- Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dodaniu nagłówków protokołu HTTP: `Content-Type: application/json`, `Content-Type: multipart/form-data`.

```
[user@parrot]~$ openssl genpkey -algorithm RSA -out priv.pem -pkeyopt rsa_keygen_bits:1024
.....
.....
[user@parrot]~$ cat priv.pem
-----BEGIN PRIVATE KEY-----
MIICeAIBADANBgkqhkiG9w0BAQEFAASCAmIwggJeAgEAAoGBANihix12eEvufy1L
TFm+YJ4xvcwvE2atpJ4JbcksA6+kwy7f7rsQkeLw181fXyT3IrHut3s0gEkyEGRA
Rt6jBn4ehAt8tESVBWdp85U7004LmvUIeZXah0gPMwLPAz8kdVBhUK03B0Rdoxxw
aPOR+u5P6VJVx9M84Q8hkgww+SSNAgMBAAECgYEAuuvhIuIHeNVoydPcJI+9xKJ8
98MjBXUNVwQhw/UD1Tog5MB6uNTaXxm/+GsMjRArrGhmdN1NqrBCXmQNMA1JnQ
WE+W6ujAJndgvRoBKwfEbbdg5j2M/12yscUkL9DYG/SH5YuDn8HpzpZSc1INBiSn
6LgRpkNXXqQMFrfSefW0CQQDz1hcq888BdzjsY8lgMbbDjTf2aUfn8E6ro/aYQKa0
Z8W/TRfU0HfcRz2PwZs8wD+OUBT1nXiLZxzHKu1ELcQ3AkEA43AHQHLJVlqXl+fQ
6rRB9N8ii5JF8RcYmMJNn+KaOrTbirkXMwvFxpMqhrSc+WtNYFpUiBZ6obJc43Xd
P3ZDwWJAUJOp+kaxXIGibPOYw1IKXFrZaHNoEKxnuw6e+bc7kwmV3l7ChlARm9hq
7ouDGjNJjWTHWFceU6rSZrq3x3T8RwJBAKL6nYe2ecayxlzpqTKlF3qVB1lC6XHZ
XdKFFSEUrfarsZ0YA6LltcUVomzDmpWdKeZ49IhMq3YBrYS16nTWl28CQCZ+Fd/
UD0wXDOM/Cpa50fh9Tre5CJMlLZXlB1aqkibHY0UEX6yrAnUhgwxNKh51UTC9Rs
K3jmA1buTj9P1+D0
-----END PRIVATE KEY-----
```

```
[user@parrot]~  
$ openssl pkey -in priv.pem -pubout -out pubkey.pem  
[user@parrot]~  
$ cat pubkey.pem  
-----BEGIN PUBLIC KEY-----  
MIGfMA0GCsQgSIb3DQEBAQUAA4GNADCBiQKBgQDYoyYsddnhL7n8pS0xZvmCeMb3M  
LxNmraSeCW3JLAOvpMMu3+67EJHi8JfNX18k9yKx7rd7NIBJMhBkQEbeowZ+HoQL  
fLRElQVg6f0V0zjuC5r1CHmV2odIDzFpTwM/ JHVQYVCtNwdEXaMccGjzkfruT+1S  
VcfTPOEPIZIMMPkkjQIDAQAB  
-----END PUBLIC KEY-----
```

```
[user@parrot]~  
$ curl -X 'POST' \  
  'http://127.0.0.1:3001/upload/public' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: multipart/form-data' \  
  -F 'file=@pubkey.pem'  
{  
  "bits": 1024,  
  "bytes": 272,  
  "error": null,  
  "is_rsa": true,  
  "type": "public"  
}
```

```
[user@parrot]~  
$ curl -X 'POST' 'http://127.0.0.1:3001/upload/private' -H 'accept: application/json' -H 'Content-Type: multipart/form-data' -F 'file=@priv.pem'  
{  
  "bits": 1024,  
  "bytes": 916,  
  "error": null,  
  "is_rsa": true,  
  "type": "private"  
}
```

3.2 Pod adresem <http://127.0.0.1:3002> działa prosty serwer HTTP udostępniający dwa endpointy: <http://127.0.0.1:3002/upload/ec/public> oraz <http://127.0.0.1:3002/upload/ec/private>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 3002:3002 --name ex2 docker.io/mazurkatarzyna/asymmetric-enc-ex2:latest
podman run -p 3002:3002 --name ex2 docker.io/mazurkatarzyna/asymmetric-enc-ex2:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 3002:3002 --name ex2 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex2:latest
podman run -p 3002:3002 --name ex2 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex2:latest
```

(b) Wygeneruj parę kluczy EC - krzywych eliptycznych (publiczny i prywatny). Wykorzystaj krzywą prime256v1. Wyeksportuj oba klucze do plików.

(c) Używając metody HTTP POST, wyślij parę kluczy EC do serwera poprzez endpointy <http://127.0.0.1:3002/upload/ec/public> (jako file) oraz <http://127.0.0.1:3002/upload/ec/private> (również jako file). W odpowiedzi serwer zwróci odpowiednią informację (szczegóły klucza).

UWAGI:

- Aby wygenerować parę kluczy EC, wykorzystaj narzędzie OpenSSL. Użyj argumentu [genpkey](#) oraz [ec](#).
- Aby wysłać odpowiedź do serwera, użyj narzędzia cURL. Pamiętaj o dodaniu nagłówków protokołu HTTP: Content-Type: application/json, Content-Type: multipart/form-data.

```
[user@parrot]~$ openssl genpkey -algorithm EC -out ecpriv.pem -pkeyopt ec_paramgen_curve:prime256v1
[user@parrot]~$ cat ecpriv.pem
-----BEGIN PRIVATE KEY-----
MIGHAgEAMBMGBYqGSM49AgEGCCqGSM49AwEHBG0wawIBAQQgCsUti4tRcHTM8SX0
Er5oTuvrWs+O1lmd2HTJxohIy5KhRANCAARMAW5sfkEMuSJI7CFD/xYSL5nFDKus
un+9wVDs5SNAdxC+Q0oH5ZBfuXIYuxcfKU/X0hZi0HSa3ZTQ88J1RCnT
-----END PRIVATE KEY-----
```

```
[user@parrot]~$ openssl ec -in ecpriv.pem -pubout -out ecpub.pem
read EC key
writing EC key
[user@parrot]~$ cat ecpub.pem
-----BEGIN PUBLIC KEY-----
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEATAFubH5BDLkiS0whQ/8WEi+TXwyr
rLp/vcFQ70UjQHcQvkNKB+WQX7lyGLsXHy1P1zoWYtB0mt2U0PPCdUQp0w==
-----END PUBLIC KEY-----
```

```
[user@parrot]~  
$ curl -X 'POST' \  
  'http://127.0.0.1:3002/upload/ec/public' \  
  -H 'accept: */*' \  
  -H 'Content-Type: multipart/form-data' \  
  -F 'file=@ecpub.pem'  
{  
  "bytes": 178,  
  "curve": "NIST P-256",  
  "error": null,  
  "is_ec": true,  
  "type": "public"  
}
```

```
[user@parrot]~  
$ curl -X 'POST' \  
  'http://127.0.0.1:3002/upload/ec/private' \  
  -H 'accept: */*' \  
  -H 'Content-Type: multipart/form-data' \  
  -F 'file=@ecpriv.pem'  
{  
  "bytes": 241,  
  "curve": "NIST P-256",  
  "error": null,  
  "is_ec": true,  
  "type": "private"  
}
```

3.3 Pod adresem <http://127.0.0.1:3003> działa prosty serwer HTTP udostępniający jeden endpoint: <http://127.0.0.1:3003/checkkeys>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 3003:3003 --name ex3 docker.io/mazurkatarzyna/asymmetric-enc-ex3:latest
podman run -p 3003:3003 --name ex3 docker.io/mazurkatarzyna/asymmetric-enc-ex3:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 3003:3003 --name ex3 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex3:latest
podman run -p 3003:3003 --name ex3 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex3:latest
```

(b) Wygeneruj parę kluczy RSA (publiczny i prywatny) o długości 1024 bitów. Wyeksportuj oba klucze do plików.

(c) Używając metody HTTP POST, wyślij parę kluczy RSA do serwera poprzez endpoint <http://127.0.0.1:3003/checkkeys> (jako pliki `private_key.pem` oraz `public_key.pem` w jednym requeście). W odpowiedzi serwer zwraca informację o poprawności i dopasowaniu kluczy (czy klucz publiczny odpowiada przesłanemu kluczowi prywatnemu).

UWAGI:

- Aby wygenerować parę kluczy RSA, wykorzystaj narzędzie `OpenSSL`. Użyj argumentu [genpkey](#) oraz [pkey](#).
- Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dodaniu nagłówka protokołu `HTTP Content-Type: application/json`.

```
[user@parrot]~$ openssl genpkey -algorithm RSA -out priv.pem -pkeyopt rsa_keygen_bits:1024
.....+++++
.....+++++
[user@parrot]~$ cat priv.pem
-----BEGIN PRIVATE KEY-----
MIICdgIBADANBgkqhkiG9w0BAQEFAASCAmAwggJcAgEAAoGBAKxJj4TOYzIpquTg
YabYf0JAShJRY9CmaeqD0Qb0BIAz8N5K249Xxvm3nbXW/zW+yOyuCdtNELAJ+5KZ
lFhFX7MDV5tbUIqz7IHUkgy54FuR7Ei2jSqQHC3XUD+6rR79311dsugaTJ3w8558
ChhAw3TKcaKiM6QyvXTiRltMquYLAGMBAAECgYBlx18dmb9IFw16AIQvyJ4FSW14
vNQ2k3z0qeiGGJbYFdcupxdr6w0E4NAuaECEQ5z3FSmFj7FwDbWsuUXFj9YkNpjU
d0qhH750B0j0Pc0GP8GmdNcE0hfbJdFgb7963h2X/eJ3J0jR0o3J0qveBOI2cTqh
7iNXh4PsyzzKpQ0uYQJBANZ27rq3Q3Bxk8xwfiSP1B4ouz5MNzU1OuWt+pvEu/4a
b05nc1f84SBGCoKurTvKw1XEQ29EYmFLw2yFKeaCCWcCQDNp39w3UPqUsv0bA0L
528SKu1bfm7/Qu3GdzIzxs49QpoI4Ls6oN0r10/kKQcyJRQ/G/pI4PyAua90r6vz
rE09AkB4j1KUUBt70h9lB7YRgWPD0Zy9pTyIw0AqNP/TBABPwrin/P+0gAbDERM
QbTPr6MuS2ezHFxRK62nCxCQKp35xAkBCsd2MttV9uthJ25Tn1WjaFX3gvEI3Gne0
c72e0Q1NpLnwzY1L082XZ+VORESXCASz3n+BKke/hk+2HIFloppAkEAoiw7i0N6
egBGGP6Vw7FeKhcu/BdNU+4uu4RCF86yVD3fXyMw1Ppe8iMNpsJ0tMt061NNSx1
bYNWWcfaFQOYZw==
-----END PRIVATE KEY-----
```

```
[user@parrot]~$ openssl pkey -in priv.pem -pubout -out pubkey.pem
[user@parrot]~$ cat pubkey.pem
-----BEGIN PUBLIC KEY-----
MIGfMA0GCsGSIb3DQEBAQUAA4GNADCBiQKBgQCsSY+EzmMyKark4GGm2H9CQEoS
UWPQjGnqg9EGzgSAM/DeStuPV8b5t5211v81vsjsrgnbZxCwCfuSmZRYRV+zA1eb
W1CKs+yB1JIMueBbkexIto0qkBwt11A/uq0e/d9dXbLoGkyd8POefAoYQMN0ynGi
ojOkMr104kZbTKrmCwIDAQAB
-----END PUBLIC KEY-----
```

```
[user@parrot]~$ curl -X 'POST' 'http://127.0.0.1:3003/checkkeys' -H 'accept: application/json' -H 'Content-Type: multipart/form-data' -F 'private_key_file=@priv.pem' -F 'public_key_file=@pubkey.pem'
{"result": "Keys are valid and match!"}
```

Notatka

Nie trzeba w żaden sposób edytować plików

3.4 Pod adresem <http://127.0.0.1:3004> działa prosty serwer HTTP udostępniający dwa endpointy: <http://127.0.0.1:3004/getprivkey> oraz <http://127.0.0.1:3004/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 3004:3004 --name ex4 docker.io/mazurkatarzyna/asymmetric-enc-ex4:latest
podman run -p 3004:3004 --name ex4 docker.io/mazurkatarzyna/asymmetric-enc-ex4:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 3004:3004 --name ex4 ghcr.io/mazurkatarzyna/asymmetric-enc-ex4:latest
podman run -p 3004:3004 --name ex4 ghcr.io/mazurkatarzyna/asymmetric-enc-ex4:latest
```

- Wyślij request do endpointa <http://127.0.0.1:3004/getprivkey> używając metody HTTP GET. Otrzymasz w odpowiedzi identyfikator sesji (w nagłówku X-Session-ID), oraz klucz prywatny RSA (`private_key.pem`).
- Na podstawie otrzymanego klucza prywatnego, wygeneruj odpowiadający mu klucz publiczny. Ilu bitowy jest klucz?
- Za pomocą metody HTTP POST, wyślij pod endpoint <http://127.0.0.1:3004/submit> wygenerowany klucz publiczny (jako plik, `public_key.pem`) oraz identyfikator sesji (`session_id`).

UWAGI:

- Aby wygenerować klucz publiczny, wykorzystaj narzędzie OpenSSL. Użyj argumentu [pkey](#).
- Aby wysłać odpowiedź do serwera, użyj narzędzia cURL. Pamiętaj o dodaniu nagłówka protokołu HTTP Content-Type: `application/json`.

```
[user@parrot]-[~]  
$ curl -D headers.txt -o privkey.pem -X GET 'http://127.0.0.1:3004/getprivkey' -H 'accept: application/json'  
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current  
           Dload  Upload   Total      Spent      Left  Speed  
100 1678  100 1678    0     0 26432      0 --:--:-- --:--:-- --:--:-- 26634
```

```
[user@parrot]-[~]  
$ cat headers.txt  
HTTP/1.1 200 OK  
Server: Werkzeug/3.1.3 Python/3.10.19  
Date: Tue, 11 Nov 2025 19:00:01 GMT  
Content-Disposition: attachment; filename=private_key_8b83a0c58a331412.pem  
Content-Type: application/octet-stream  
Content-Length: 1678  
Cache-Control: no-cache  
Date: Tue, 11 Nov 2025 19:00:01 GMT  
X-Session-ID: 8b83a0c58a331412  
Connection: close
```

```
[user@parrot]~  
$ cat privkey.pem  
-----BEGIN RSA PRIVATE KEY-----  
MIIEpAIBAAKCAQEA0mKjGWI6x5+KdcvPL+kSy2LBVsb9hGFXUNGx4ybMomxQ2cg5  
brxA94pfTv1r/397aluxImWFzXzbaKHAd+18RIBwJMPyzRXvL/dIm0fYXNyDLmIe  
m2K2tZbY2R2xxJbTRM9i0arqg1XpnM2nwqgQwyArHDeP57H2dfkWPpSaQG4CRuwU  
HMs9iislhDaG2yV6kd+Cc3uy7CpDypV7ehNkSK9jYjKnt72GAs5i0PeA3ZEqypdM  
7Tl7a0NwcD64P94mDwQ/J/07XQutcpaT/+WPX1RzCpidSxwCgt08VSPvmKwkdbAS  
EKpXN1Ab25byPWb9VArfdqeuMuJp92MBXYwE1wIDAQABAOIBAefFGGivZuf0mBJp  
TavFcgDqFpSY26F0Z7ANaQbZlJOQKeCxeK45cFxB2x0iRjZp8dstnnbidUvNXTg  
LhvDEl1D0E82m/uxRoEAXtrAqSTgcja/EDzbo7PVzmVQv2hX7ceUWblqiWNAm/cPd  
MG6qUNWP4sUEe1YKEs7aA2sBMMHrZtkB/dfDVPtsDuM+BS/9tTIS4tQ7KgygCVvo  
yiW9AQINT4KtqpuUceICQX04BFSiCMwPg69Y0XzyFA2tKqNPUi37n3Z50gPSZ3Bk  
FZyqJ1aidd65dscSuciwl75poeaKp3VVhsu8vV9pmDVDk7mfQoRzCGx76Ds7xZ/K  
Ob1EG/ECgYEA30bnRplfRHCqY+e6EC/FpSxZZ2urVyc15z8lnVamwM0g+yrnk6Wz  
wPwDx74q6swHjV58ujtKT7e6j++SEq9Z5lpXF60UXehdgZG/po/DfE1XmmSPeHRM  
ImeFhc8+2bX/eUOb7AsK1zic0dB8p1Q/47sTo81NIeobW+R33B+H70kCgYEA88/4  
/80BLTmwo5Kyb8DPnJq18nW6UwvzxRPpTuy2nLEbW2FKuFwz6fAbN12560fjJ6Ix  
URXBIUB34sLeFizc56XFIGcNFsgGLWdPERBFkjg5L1at+AbmCKveAAyo3oIyfc  
hpXjZQ7M7KpxeZBRknU9DCC/Pev3SHs2EMI0KqS78CgYEAzNnYKYs0PvyYFn0jCTWz  
/LuG4TBowTaNomffcuwTUYIojqM4A6TLcDCEtOu2Uu0IJ+IymmU3QXvpJ2eMN4Wj  
QcoPL+aSl49Pf0QZb50E2yJy6Rro9F9dP45RWBuMWjkVCkNZDTNRc67JptHrNs+  
MEqhK0rPF59I5cuWF2yqGVkCgYBH8++LI5piy2y6yVdJrcBXej0KAf5r9dIKLk+V  
NL+LvKTTMgJIKEXV21tcjwqHlq+uCaq+QEpP0uP0ky3Ro4nJ/100s/rj50ITcjKe  
WYv+nAMcWlbtIEYQ9w+80BUlnYzjHBtDONJnIJwj9c3d+E9n5+rsdhsdq3lE0fYv  
w1VhpwKBgQCuzMEhgXkV0BoK6s3bDF11hlGKzq6WP2ag3NbvcG4Ixf2cqfDi/fzk  
F7ibfpaI1B5eqaSQBARKb1JL8EuhTii/KvMY1kMWqUVQy/SlrhlosxdCzA7B/1Ra  
yZC0eKE1u+u9UpFDItqMkTI2xT8S2VJ6sT1zax1z65KN2r7W2P7U4Q==  
-----END RSA PRIVATE KEY-----[user@parrot]~
```



```
└─ $openssl pkey -in privkey.pem -pubout -out pubkey.pem
└─ [user@parrot]~$
└─ $cat pubkey.pem
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA0mKjGWI6x5+KdcvPL+kS
y2LBVsb9hGFXUNGx4ybMomxQ2cg5brxA94pfTv1r/397aluxImWFzXzbaKHad+18
RIBwJMPyzRXvL/dIm0fYXNyDLmIem2K2tZbY2R2xxJbTRM9i0arqg1XpnM2nwqgQ
wyArHDeP57H2dfkWQpSaQG4CRuwUHMs9iislhDaG2yV6kd+Cc3uy7CpDypV7ehNk
SK9jYjKnt72GAs5i0PeA3ZEQypdM7Tl7aONwcD64P94mDwQ/J/07XQutcpaT/+WP
X1RzCpidSxwCgt08VSPvmKwkdbASEKpXN1Ab25byPWb9VArfdqeuMuJp92MBXYwE
1wIDAQAB
-----END PUBLIC KEY-----
```

```
└─ [x]~[user@parrot]~$
└─ $curl -X 'POST' \
    'http://127.0.0.1:3004/submit' \
    -H 'accept: application/json' \
    -H 'Content-Type: multipart/form-data' \
    -F 'session_id=8b83a0c58a331412' \
    -F 'public_key_file=@pubkey.pem'
{"result": "Keys match!"}
```

3.5 Pod adresem <http://127.0.0.1:3005> działa prosty serwer HTTP udostępniający dwa endpointy: <http://127.0.0.1:3005/getprivkey> oraz <http://127.0.0.1:3005/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 3005:3005 --name ex5 docker.io/mazurkatarzyna/asymmetric-enc-ex5:latest
podman run -p 3005:3005 --name ex5 docker.io/mazurkatarzyna/asymmetric-enc-ex5:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 3005:3005 --name ex5 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex5:latest
podman run -p 3005:3005 --name ex5 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex5:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:3005/getprivkey> używając metody HTTP GET. Otrzymasz w odpowiedzi identyfikator sesji (w nagłówku X-Session-ID), oraz klucz prywatny EC (private_key_pem).
- (c) Na podstawie otrzymanego klucza prywatnego, wygeneruj odpowiadający mu klucz publiczny. Ilu bitowy jest klucz?
- (d) Za pomocą metody HTTP POST, wyślij pod endpoint <http://127.0.0.1:3005/submit> wygenerowany klucz publiczny (jako plik, public_key_pem) oraz identyfikator sesji (session_id).

UWAGI:

- Aby wygenerować klucz publiczny, wykorzystaj narzędzie OpenSSL. Użyj argumentu [ec](#).
- Aby wysłać odpowiedź do serwera, użyj narzędzia curl. Pamiętaj o dodaniu nagłówka protokołu HTTP Content-Type: application/json.

```
[user@parrot]-[~]
└─$ curl -D headers.txt -o privkey.pem -X GET 'http://127.0.0.1:3005/getprivkey' -H 'accept: application/json'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload  Total  Spent    Left  Speed
100  240  100  240    0    0 10698      0 --:--:-- --:--:-- --:--:-- 10909
[user@parrot]-[~]
└─$ cat headers.txt
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.10.19
Date: Tue, 11 Nov 2025 19:07:08 GMT
Content-Disposition: attachment; filename=private_key_1148810abe07898c.pem
Content-Type: application/octet-stream
Content-Length: 240
Cache-Control: no-cache
Date: Tue, 11 Nov 2025 19:07:08 GMT
X-Session-ID: 1148810abe07898c
Access-Control-Allow-Origin: *
Connection: close

[user@parrot]-[~]
└─$ cat privkey.pem
-----BEGIN PRIVATE KEY-----
MIGHAgEAMBMGBYqGSM49AgEGCCqGSM49AwEHBG0wawIBAQQGpYWhidG7nXOBB+g
tIsHpsEKVkpq22NyPGmK8RF3Kx+hRANCAARscWBvJENgv8PR98cInLVniPo9FP1
Jtw1J/DNVF3bqRmfBE0GV0m0YJ1Dp9WX5u1NSY8ipF030xGKx7q10na0
-----END PRIVATE KEY----- [user@parrot]-[~]
```

```

$openssl pkey -in privkey.pem -pubout -out pubkey.pem
[user@parrot]~$
$cat pubkey.pem
-----BEGIN PUBLIC KEY-----
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEa7HFgbyRDYL/D0ffHCJy1Z4j6PRT
5SbcNSfwzVRd26kZnwRNBldJtGCdQ6fVl+btTUmPIqRdN9MRise6tdJ2tA==
-----END PUBLIC KEY-----
[user@parrot]~$
$curl -X 'POST' \
'http://127.0.0.1:3005/submit' \
-H 'accept: application/json' \
-H 'Content-Type: multipart/form-data' \
-F 'session_id=1148810abe07898c' \
-F 'public_key_file=@pubkey.pem'
{"result": "Keys match!"}

```

3.6 Pod adresem <http://127.0.0.1:3006> działa prosty serwer HTTP udostępniający 2 endpointy: <http://127.0.0.1:3006/encrypt> oraz <http://127.0.0.1:3006/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```

docker run -p 3006:3006 --name ex6 docker.io/mazurkatarzyna/asymmetric-enc-ex6:latest
podman run -p 3006:3006 --name ex6 docker.io/mazurkatarzyna/asymmetric-enc-ex6:latest

```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```

docker run -p 3006:3006 --name ex6 ghcr.io/mazurkatarzyna/asymmetric-enc-ex6:latest
podman run -p 3006:3006 --name ex6 ghcr.io/mazurkatarzyna/asymmetric-enc-ex6:latest

```

- (b) Wyślij request do endpointa <http://127.0.0.1:3006/encrypt> używając metody HTTP GET. Otrzymasz w odpowiedzi unikalny identyfikator sesji (w nagłówku X-Session-ID), losowe słowo do zaszyfrowania (w nagłówku X-Word) oraz klucz publiczny RSA w formacie PEM (jako `public_key.pem`).
- (c) Zaszyfruj otrzymane słowo, stosując algorytm RSA-2048 z użyciem klucza publicznego oraz wybranego trybu paddingu: OAEP.
- (d) Wyślij request do serwera poprzez endpoint <http://127.0.0.1:3006/submit> używając metody HTTP POST, identyfikator sesji (`session_id`), jak i zaszyfrowane słowo (jako plik, `encrypted_file`).
- (e) W odpowiedzi serwer zwróci odpowiednią informację o sukcesie lub błędzie - zweryfikuje poprawność przesłanego zaszyfrowanego i zakodowanego słowa.

UWAGI:

- Aby zaszyfrować odebrane od serwera słowo, wykorzystaj narzędzie OpenSSL. Użyj argumentu [pkeyutl](#).
- Aby wysłać odpowiedź do serwera, użyj narzędzia cURL. Pamiętaj o dodaniu nagłówka protokołu HTTP Content-Type: `application/json`.

```
[user@parrot]~$ curl -s -D data.txt -o key.pub -X get http://localhost:3006/encrypt
[user@parrot]~$ cat data.txt
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.10.19
Date: Fri, 21 Nov 2025 16:22:09 GMT
Content-Disposition: attachment; filename=public_key_qAFbZePdx8M.pem
Content-Type: application/octet-stream
Content-Length: 450
Cache-Control: no-cache
X-Session-ID: qAFbZePdx8M
X-Word: bhea16
Access-Control-Allow-Origin: *
Connection: close

[user@parrot]~$ openssl pkeyutl -encrypt -pubin -inkey key.pub -pkeyopt rsa_padding_mode:oaep -in <(echo -n "bhea16") -out encrypted.bin
```

```
[user@parrot]~$ curl -X 'POST' 'http://localhost:3006/submit' -H 'accept:application/json' -H 'Content-Type: multipart/form-data' -F 'session_id=qAFbZePdx8M' -F 'encrypted_file=@encrypted.bin'
{"result": "Correct encryption!"}
```

◆ ZASZYFROWANIE SŁOWA RSA-2048 + OAEP

Użyj narzędzia `openssl pkeyutl` z trybem paddingu OAEP.

```
echo -n "slowo" | openssl pkeyutl -encrypt -pubin -inkey pubkey.pem -pkeyopt rsa_padding_mode:oaep -out encrypted.bin
```

💡 Co tu się dzieje:

- `echo -n "student2025"` — wprowadza słowo do szyfrowania
- `-encrypt` — tryb szyfrowania
- `-pubin` — klucz publiczny
- `-inkey pubkey.pem` — ścieżka do klucza publicznego
- `-pkeyopt rsa_padding_mode:oaep` — tryb paddingu OAEP
- `-out encrypted.bin` — wynik zaszyfrowany zapisany binarnie

3.7 Pod adresem <http://127.0.0.1:3007> działa prosty serwer HTTP udostępniający dwa endpointy: <http://127.0.0.1:3007/decrypt> oraz <http://127.0.0.1:3007/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 3007:3007 --name ex7 docker.io/mazurkatarzyna/asymmetric-enc-ex7:latest
podman run -p 3007:3007 --name ex7 docker.io/mazurkatarzyna/asymmetric-enc-ex7:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 3007:3007 --name ex7 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex7:latest
podman run -p 3007:3007 --name ex7 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex7:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:3007/decrypt> używając metody HTTP GET. Otrzymasz w odpowiedzi unikalny identyfikator sesji (w nagłówku `X-Session-ID`), klucz prywatny oraz zakodowane i zaszyfrowane słowo, oba zapakowane w pliku *.zip. (Serwer wygenerował losowe słowo, zaszyfrował je algorytmem RSA (klucz 4096-bit) i zakodował w formacie base64.)
- (c) Odkoduj podane słowo, następnie odszyfruj je przy użyciu klucza prywatnego i algorytmu RSA-4096 z paddingiem OAEP.
- (d) Wyślij request do serwera poprzez endpoint <http://127.0.0.1:3007/submit> używając metody HTTP POST, identyfikator sesji (`session_id`) oraz odszyfrowane słowo (`decrypted_word`).
- (e) W odpowiedzi serwer zweryfikuje poprawność odszyfrowanego słowa i zwróci informację o sukcesie lub błędzie.

UWAGI:

- Aby odszyfrować odebrane od serwera słowo, wykorzystaj narzędzie OpenSSL. Użyj argumentu [pkeyutl](#).
- Aby wysłać odpowiedź do serwera, użyj narzędzia cURL. Pamiętaj o dodaniu nagłówka protokołu HTTP Content-Type: application/json.

◆ Krok 1 — Uruchom serwer

```
docker run -p 3007:3007 --name ex7 docker.io/mazurkatarzyna/asymmetric-enc-ex7:latest
```

SHELL

lub:

```
podman run -p 3007:3007 --name ex7 docker.io/mazurkatarzynaumcs/asymmetric-enc-ex7:latest
```

SHELL

♦ Krok 2 — Pobierz dane z serwera (/decrypt)

Jedno polecenie, które pobiera ZIP i zapisuje nagłówki (z `session_id`)

```
curl -s -D headers.txt -o response.zip -X GET 'http://127.0.0.1:3007/decrypt' -H 'accept: application/json'
```

SHELL

♦ Co robi każda opcja:

- `-s` — tryb “silent” (bez pasków postępu),
- `-D headers.txt` — **zapisuje wszystkie nagłówki HTTP** do pliku `headers.txt`,
- `-o response.zip` — **zapisuje treść odpowiedzi (plik ZIP)**,
- reszta — Twój standardowy request.

```
[x]-[user@parrot]-[~]  
$ curl -s -D headers.txt -o response.zip -X GET 'http://127.0.0.1:3007/decrypt' -H 'accept: application/json'
```

📁 Po tym poleceniu masz dwa pliki:

headers.txt
response.zip



🔍 Odczytaj `session_id` z nagłówków:

```
grep -i X-Session-Id headers.txt
```

SHELL

Przykład wyniku:

```
X-Session-Id: 4dbd46f4265e5e03
```

➡ to jest Twój `session_id` do późniejszego POST-a.

```
[user@parrot]-[~]  
$grep -i X-Session-Id headers.txt  
X-Session-Id: ffb816a2eee8890b
```

```
[user@parrot]-[~]  
$cat headers.txt  
HTTP/1.1 200 OK  
Server: Werkzeug/3.1.3 Python/3.10.19  
Date: Tue, 11 Nov 2025 14:29:48 GMT  
Content-Disposition: attachment; filename=encrypted_and_key.zip  
Content-Type: application/zip  
Content-Length: 4156  
Cache-Control: no-cache  
Date: Tue, 11 Nov 2025 14:29:48 GMT  
X-Session-Id: ffb816a2eee8890b  
Access-Control-Allow-Origin: *  
Connection: close
```

◆ Krok 3 — Rozpakuj plik ZIP

```
unzip response.zip -d zad7
```

SHELL

Po rozpakowaniu zobaczysz coś takiego:

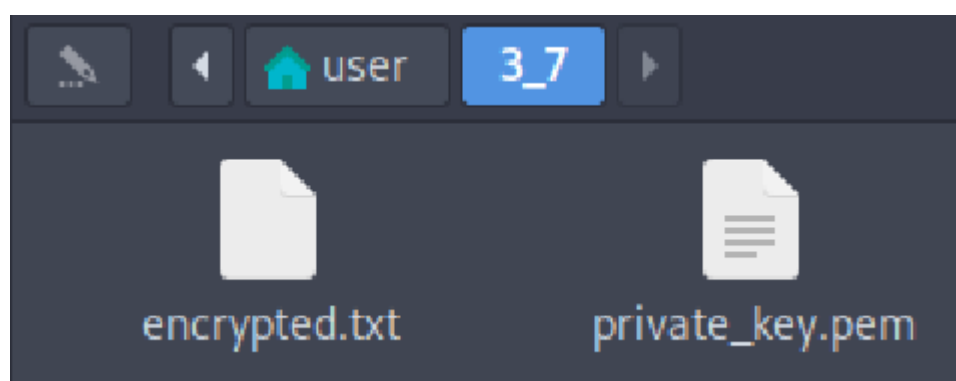
```
ls zad7
```

SHELL

Wynik:

```
encrypted.txt  
private_key.pem
```

```
[user@parrot]-[~]
$ unzip response.zip -d 3_7
Archive:  response.zip
  extracting: 3_7/encrypted.txt
  extracting: 3_7/private_key.pem
[user@parrot]-[~]
$ ls 3_7
encrypted.txt  private_key.pem
```



◆ Krok 4 — Sprawdź, co jest w pliku **encrypted.txt**

```
cat zad7/encrypted.txt
```

SHELL

Zobaczysz długi ciąg Base64 — to zaszyfrowane dane (RSA4096 + OAEP).

```
[user@parrot]-[~]
$ cat 3_7/encrypted.txt
awojFGVM/X190bQmM0cXFQj7foFaiW0GLFQ+iBf1gzRx2EgnGjp0D/kKwBUTuEsu0CwyNa1TKC3jcppMk0ots1XGnFMwY0rk+iIg8mZCLPYh9Qh8iR4/meQvkZp1X5EMeJMZE/ZBzNDKxI1GxEbv/kJ/2I1ZuM/siraf
gkg5qG0/KsQW07fVosbFT1sImj7RTBp1st4YuqW6Gjz11DwB7bPGQeLIR5LheLQGY4RU6+C9S76ZdIqDjtZ5gF/JJKya5RfRKnMnJNhTxBYWS11JIHy89u/MTKzhjnm9mAUpsZ9bAR9cztig5rav0+vFPt/vPvTeEHL+5
GB/8ZqboBD1JGVJFEg/NEZ0s3+KSOE3prxc3nMlu4KD1I61zprx9At1QBp3SRpvJZ1BHFw16BKGBTWAIcFhd+Vtm/7qcu5fA8zpmNZ0SJpLmie5ws9fyaCgyxzNG+Vh8VBmnb4eg0p1UCeITaZ4Vn4TW16PSGYREvtzCS
4oWkbtWkaComZ/fezi29isTjcAu2ZN6083D4EbA86d05MB65imdte0ZE3Ny8HY0w6rbfumJua5SKznGN6Uvp08ZceZ/jW7zuyKb+Z15/AAZEffshRWfJ0QJ/Vh7j6kaUkyoaIKCbrxUgwYaSa+D16faBmR9yqVn8dLhT
RwadoHx7J07+FMctvc8=[user@parrot]-[~]
```


Dodatkowo private_key.pem

```
$cat 3_7/private_key.pem
-----BEGIN RSA PRIVATE KEY-----
MIIEKAAIBAAKCAgEAt1kHa8UFHgddsn9muG+vEoun9pAf9HGtb+QARSk7P+LyhOzE
bZ3gWQccKUJ/e3PN6MoiM+txnuuIBOZO1l3opwWENCyPYs0xo+wLkRcHFkP13VFI
2eGK0UrrQqMPPfORF3CqQqMLfWyWRpyy0psyCzvEUB1WKvx8vevMeZ1kdhmRLacoL
Rg39Z7cPBTY2xxpnZxwnk1Q1mpLoBY2JYXb+Xg50hz+4BJGD4urEqx3UP5m8i+mj
dvg6CgAUvZ8b1E9jxG9x0J76JZmDKtuwXbdSrG1SWXk7BcFo19d1J/8DAI3vWsHn
gjr7sb3exD8sou5D+8V1DJWKHpjRR3g/71ZpR+KObYsbzu0NcRUik+qab4FmS0aX
4sNsCnjJ7dv/0YFPRATy5BTIYe0wZqSI88q/Ezh/1+y4KT58ftWUiOwcw3fvHVjN
701tynhYIFE7j91uuhgRCdl5boP185XPSLNSwn/hw30+8xbpINgehj4tQ1DkFEIK
U07uoLQtfsUE3aKn0L4Hq94UzBiduFcsDGtz6h3sXtjWi/IMhuAvw1LO05bZNER4
qKmC81wb56KgzzU/vmERbwWnJ2SXUUSvYc8TyXX8wbMOe0+sb247JH7mrkEtz7jk
0h4UmbTDhAz/IgIKjdm0clQYzW0eawJdRRap5NDc5Fq3GzS/t1A/Yp/zIS8CAwEA
AQKCAgAODRIfBbnXK8xX9iCKqPB7VN2Z7UCtrJqULR4aLsc+JafdXQH0vscn037C
gLihXuYcXYVSgkwyWC2T5oyGRIskZa1UjEeu1DfmLpxSUbf20lxG4khZfknzsS6d
iGgZ06qw7UgpzslsRExNygTMNaceseYNeDfKXtr2KrSdN14nYv90i6/ifgb6EFLj
3NxE1Pawtb0ony9Ekdab01YzdJOY5LLK9aymGPrJBKHu1TnjZ+ZFuLGQ2AQmqQvt
3kgwTQdB0iKgUNat0/5z5eM1w2tgEH67oa10zxw4UmhFm0sDe10pEOwrk9cY74qn
con6sJTgMigE117+6ko7lnEb1F58BSE/0agAVoscR9fkc3dK2EfrC8Qkz3uqZl0b
0UC+4x/0t9sSb7Cr5aZvy8qJLpbcTk3jgjh3ax7/5hv3y2SRmI5Ak0s7cZkqGca4
d5fvndwq115oM9DhucQPoc9J9bWx+q40Uhxh0UksyKP5XdWLSefiMrpC1dN4aHHU
Ieq4wH4ZmZ2BrSdv0vfa7v1VsjjUYEBG0f4nR7F72A1ZJ3CqFpZi3LyE+c8QobyQ
rkg235Nnn0Ib85SXojDAK36NTw30soLUVymijgx/VBuU077VYRJfa4oWmIjZiy50
2nJK1+UfFUcAwHAY1vZm86k91m6IVCBvbLM80Z9T+CvqD1XU9QKCAQEA14XEPSsH
Qh+QB1sX+VfMHTCm2fXpvSFLJV90BwskfVb0qzHs4nucuPecI8n3zqeg+67VR4BW
vypU3JHC4I1RrwGoQx0xjCcikmE2XafrUpqPMr5qoezummdgq3wBMT8/4EqTw2sS
A7pqycvr0XkMo8aHLiRLb75zDPu55/q4uV0FQmSLgxXnKIWBtgPi9NvX/1upMcem
vUi0kHsJNMYSnzi6R/vXQW1YijaDNqc3ISVTmNOF4enHQzfwPD6IoJumBvCi/QCz
LOFjy9GUEySS1RZzdWRioAW5trgSyFW9CpuzN4Q4r1eCJmLNOFHaKeRC3jbQgu5i
eRvsqjpUqU6q4wKCAQEA2chQQ3mpcJiQMfbMAS0c4E9Di5ZzETn1YPLEVSYCHhna
J5V5B+MG/Z2DAUbsot2GAXoJ2ib+rzZIMVcVfW+JlPH8sJMQbCAPmd/c3Ir+FSYK
tLw6vhTelOFHZWxk5IwSvAXX7UCxQYPhKTjW6ywWGjwaSURQXkV1Sri8PKR5gvnm
VWDZhgjcm4UD5Z9MEcXWYLEnTa/y2TvUegB0rK9MgxV06JBnx1iuZYxG0r4g/xBK
E40AGt5e0uOnqnCNdZFBdpWE1MpTx/1PCah4Uh3ZEDQvSVzm75KNUWMiujfUbgfe
df82ys6iUHZDL4ryPjrNAonCveH6AgxZaUTIHvdGRQKCAQEApuYyc1vCKGP+2fEa
FSLSqXy8LCXhTS7w7Mwm0JTCOuPOhR04m4jFb1RzTDvGtZh57y9r43g6kEp0ZhHh
```

```
zsp04047eKY2b1FAwdaP1QuKgIOM2xWSwXjw1DvDfCMS4wuo2UU5ycMgQP3Ek9u0
IROHr0FAQxKRTTZcFZQoRRVWgs0mkIJ/Q14V0k4Ta1/VvYm0Q+Wwgj6n0zu1Sb8k
E3brp3Uh4r30nfjY1MD0qIFd1i1Dzc5GPDxKmXKY7gvkTJK0p1P/H19gDUcdfKDU
mk4N1CNxEJOa1TMu+kZ6GW8xG6bJsQXXWByc9QYIcm7NDh5FbxvJlvJxUoQpY3Jz
pptwkwKCAQBw9rB7b33uQ5R39jX3XjkeOL8mhpGFG/UlpockS6TJjEtWQPgYmTA1
GTC81tcj25gI6A54M8XjiFqGDU++Ur6gnayewonALBkrjaz4ULheq+7dtM0bj60s
wfxQrBrhWa7WDInmLKpE+20tfqgsMB/+17Ii0uaFH6Xczl0FcqL8xzcsjiIGnk3q
A1L0J17E0UPTv9BNQbjL6XbbGaVrdTpQvC55+315L4y2a7559ekTpJVn7cvjI/jo
vvFXPS6DDycZs4DFB3XRxJjcv6A5p/XRq9/jQ+cd2Xzd7ogy44ay1C4WMP1vMSOM
b0uIm5/vm3x/lomHCNIvgcxXCDBtr4GpAoIBACetoy8G6uBLNdZFqETB4PGGuNeJ
ZICK02vlnaCH1eRTFdScNAcNucUYbEcKM9MF2ZnpSs7hg5SJguv7ikKQ+jj2qdqa
ah0ttX45P47QBzv0JbknS3pDUyfycSVQx8Q6AA0HrA7SuG8uW+xw1wAGv0shYvSR
0EbKT1aN82Wz9u0e4f2u7wR5jGMQtgS4cNKgM9GLNbrR58Rf3AKC4FDed44f6fNk
+KMY1NJRUxgGRetXXAo9damDwmskMKLx0Xw10XgMFzpuFr3uc4U+h3WU1/stU7Ow
OHs1q0XPPkQKerjyTQSN8JbrWhziC805Da0nHvF7/fMy2ex5Z5syTbmyKTS=
-----END RSA PRIVATE KEY----- [user@parrot]-[~]
```

◆ Krok 5 — Odszyfruj słowo RSA-4096 OAEP

Użyj **OpenSSL** z argumentem `pkeyutl` (jak w poleceniu z uwag):

```
openssl pkeyutl -decrypt -inkey zad7/private_key.pem -in <(base64 -d
zad7/encrypted.txt) -pkeyopt rsa_padding_mode:oaep -out decrypted.txt
```

SHELL

💡 Co się tu dzieje:

- `-decrypt` — tryb odszyfrowania
- `-inkey zad7/private_key.pem` — klucz prywatny RSA 4096
- `base64 -d` — dekoduje dane z base64
- `-pkeyopt rsa_padding_mode:oaep` — wymusza tryb OAEP
- `-out decrypted.txt` — zapisuje wynik do pliku

```
[user@parrot]-[~]
$ openssl pkeyutl -decrypt -inkey 3_7/private_key.pem -in <(base64 -d 3_7/encrypted.txt) -pkeyopt rsa_padding_mode:oaep -out decrypted.txt
[user@parrot]-[~]
$ cat decrypted.txt
gochargers [user@parrot]-[~]
```

lub

Krok 5 (odszyfrowanie RSA-4096 z OAEP) można zrobić **prościej**, rozbijając go na kilka bardziej intuicyjnych komend (zamiast jednego długiego z `process substitution`). Zrobimy to w **3 prostych krokach**, które łatwiej zrozumieć i debugować 🙌

♦ Wersja uproszczona odszyfrowania (zamiast jednej skomplikowanej komendy):

🧩 1 Dekoduj dane Base64 do pliku binarnego:

```
base64 -d zad7/encrypted.txt > encrypted.bin
```

SHELL

🔍 Teraz masz „surowe” zaszyfrowane dane binarne w pliku `encrypted.bin`.

```
$base64 -d 3_7/encrypted.txt > encrypted.bin
```

🧩 2 Użyj klucza prywatnego do odszyfrowania RSA-4096 z OAEP:

```
openssl pkeyutl -decrypt -inkey zad7/private_key.pem -in encrypted.bin -pkeyopt  
rsa_padding_mode:oaep -out decrypted.txt
```

SHELL

```
[x]-[user@parrot]-[~]  
$openssl pkeyutl -decrypt -inkey 3_7/private_key.pem -in encrypted.bin -pkeyopt rsa_padding_mode:oaep -out decrypted.txt
```

♦ Podsumowanie krótszej wersji:

```
# Dekodowanie Base64  
base64 -d zad7/encrypted.txt > encrypted.bin  
  
# Odszyfrowanie RSA-4096 z OAEP  
openssl pkeyutl -decrypt -inkey zad7/private_key.pem -in encrypted.bin -pkeyopt  
rsa_padding_mode:oaep -out decrypted.txt
```

SHELL

♦ Krok 6 — Wyświetl odszyfrowane słowo

```
cat decrypted.txt
```

SHELL

Wynik będzie czymś takim:

```
network2025
```

Zachowaj to słowo — to **decrypted_word**.

```
[user@parrot]~  
└─$ cat decrypted.txt  
gochargers [user@parrot]~
```

◆ Krok 7 — Wyślij rozwiązanie na `/submit`

Z odpowiedzi `curl` z `/decrypt` (tej pierwszej) zapisz również **Session-ID** z nagłówka `X-Session-ID`.

Założmy, że był to np. `7d2f9b88c31e`.

Wyślij wynik na `/submit`:

```
curl -X POST http://127.0.0.1:3007/submit \  
-H "accept: application/json" \  
-H "Content-Type: application/json" \  
-d '{  
  "session_id": "7d2f9b88c31e",  
  "decrypted_word": "network2025"  
}
```

SHELL

```
└─$ curl -X 'POST' \  
  'http://127.0.0.1:3007/submit' \  
-H 'accept: */*' \  
-H 'Content-Type: application/json' \  
-d '{  
  "session_id": "ffb816a2eee8890b",  
  "decrypted_word": "gochargers"  
}'  
{"result": "Correct decryption!"}
```

```
# 1. Uruchom serwer
docker run -p 3007:3007 --name ex7 docker.io/mazurkatarzyna/asymmetric-enc-ex7:latest

# 2. Pobierz dane
curl -i -X GET http://127.0.0.1:3007/decrypt -H "accept: application/json" -o response.zip

# 3. Rozpakuj ZIP
unzip response.zip -d zad7

# 4. Sprawdź pliki
ls zad7
cat zad7/encrypted.txt

# 5. Odszyfruj (RSA-4096 OAEP)
openssl pkeyutl -decrypt -inkey zad7/private_key.pem -in <(base64 -d zad7/encrypted.txt) -pkeyopt rsa_padding_mode:oaep -out decrypted.txt

# 6. Wyświetl odszyfrowane słowo
cat decrypted.txt

# 7. Wyślij odpowiedź
curl -X POST http://127.0.0.1:3007/submit \
-H "accept: application/json" \
-H "Content-Type: application/json" \
-d '{
  "session_id": "TUTAJ_SESSION_ID",
  "decrypted_word": "TUTAJ_TEKST"
}'
```

3.8 Pod adresem `http://127.0.0.1:3008` działa prosty serwer HTTP udostępniający jeden endpoint: `http://127.0.0.1:3008/decrypt`.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 3008:3008 --name ex8 docker.io/mazurkatarzyna/asymmetric-enc-ex8:latest
podman run -p 3008:3008 --name ex8 docker.io/mazurkatarzyna/asymmetric-enc-ex8:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 3008:3008 --name ex8 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex8:latest
podman run -p 3008:3008 --name ex8 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex8:latest
```

- (b) Wygeneruj parę kluczy RSA (jako `keys.pem`).
- (c) Wyeksportuj klucz publiczny z pary kluczy do pliku (jako `public_key.pem`).
- (d) Wybierz słowo i zapisz je do pliku (jako `plaintext.txt`).

- (e) Zaszzyfruj wybrane przez siebie słowo (`plaintext.txt`) wygenerowanym kluczem publicznym (`public_key.pem`). Wykorzystaj padding OAEP.
- (f) Zakoduj zaszyfrowane słowo i zapisz je do pliku (`ciphertext.txt`).
- (g) Wyślij request do endpointa <http://127.0.0.1:3008/decrypt> używając metody HTTP POST i prześlij do serwera:
- wygenerowany klucz prywatny (jako plik `private_key.pem`),
 - wygenerowany klucz publiczny (jako plik `public_key.pem`),
 - wybrane przez siebie słowo (jako plik `plaintext`),
 - wybrane przez siebie słowo zaszyfrowane kluczem publicznym (`public_key.pem`) z paddingiem OAEP i zakodowane w base64 (jako plik `ciphertext.txt`).
- (h) Zadaniem serwera jest sprawdzenie, czy zaszyfrowane słowo zostało zaszyfrowane podanym kluczem, oraz, czy słowa zgadzają się po odszyfrowaniu.

UWAGI:

- Aby zaszyfrować losowe słowo, wykorzystaj narzędzie `OpenSSL`. Użyj argumentu [pkeyutl](#).
- Aby wysłać odpowiedź do serwera, użyj narzędzia `cURL`. Pamiętaj o dodaniu nagłówka protokołu `HTTP Content-Type: application/json`.

```
[user@parrot]-[~]
└─$ openssl genkey -algorithm RSA -out privkey.pem -pkeyopt rsa_keygen_bits:1024
.....+++++
....+++++
[user@parrot]-[~]
└─$ openssl pkey -in privkey.pem -pubout -out pubkey.pem
[user@parrot]-[~]
└─$ echo -n "parrot" > plaintext.txt
[user@parrot]-[~]
└─$ openssl pkeyutl -encrypt -pubin -inkey pubkey.pem -in plaintext.txt -pkeyopt rsa_padding_mode:oaep -out ciphertext.txt
```

```
[x]-[user@parrot]-[~]
└─$ base64 ciphertext.txt > ciphertext.b64
```

```
[user@parrot]-[~]
└─$ curl -X 'POST' \
'http://127.0.0.1:3008/decrypt' \
-H 'accept: application/json' \
-H 'Content-Type: multipart/form-data' \
-F 'private_key=@privkey.pem' \
-F 'public_key=@pubkey.pem' \
-F 'plaintext=@plaintext.txt' \
-F 'ciphertext=@ciphertext.b64'
{"decrypted":"parrot","keys_match":true,"plaintext_match":true,"result":"ok"}
```

3.9 Pod adresem `http://127.0.0.1:3009` działa prosty serwer HTTP udostępniający jeden endpoint: `http://127.0.0.1:3009/sign`.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 3009:3009 --name ex9 docker.io/mazurkatarzyna/asymmetric-enc-ex9:latest
podman run -p 3009:3009 --name ex9 docker.io/mazurkatarzyna/asymmetric-enc-ex9:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 3009:3009 --name ex9 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex9:latest
podman run -p 3009:3009 --name ex9 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex9:latest
```

- (b) Wyślij request do endpointa `http://127.0.0.1:3009/sign` używając metody HTTP GET. Otrzymasz w odpowiedzi unikalny identyfikator sesji (`session_id`), klucz prywatny RSA (`private_key_pem`) oraz słowo (`word`).
- (c) Podpisz otrzymane słowo (`word`) kluczem prywatnym (`private_key_pem`), a następnie zakoduj wynik do formatu `base64`. Przy podpisywaniu należy zwrócić uwagę na parametry paddingu PSS:
- Używana funkcja skrótu: SHA-256
 - `Salt length`: długość soli powinna odpowiadać długości skrótu (32 bajty dla SHA-256), aby podpis był zgodny z weryfikacją po stronie serwera.
- (d) Używając metody HTTP POST, wyślij do serwera poprzez endpoint `http://127.0.0.1:3009/submit` podpisane słowo (jako `signature_b64`) oraz identyfikator sesji (`session_id`).
- (e) Serwer zweryfikuje podpis przy użyciu klucza publicznego i zwróci odpowiednią informację o sukcesie lub błędzie.

KROK 1 — uruchomienie serwera

Uruchom kontener (jeden z poniższych, w zależności od tego czy masz Docker Hub czy GHCR):

```
docker run -p 3009:3009 --name ex9 docker.io/mazurkatarzyna/asymmetric-enc-ex9:latest
```

SHELL

lub

```
docker run -p 3009:3009 --name ex9 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex9:latest
```

SHELL

KROK 2 — pobranie danych od serwera

Pobierasz klucz prywatny (`private_key.pem`), identyfikator sesji (`session_id`) i słowo (`word`).

```
curl -i -X GET 'http://127.0.0.1:3009/sign' \  
-H 'accept: application/json' \  
-o response.pem > headers.txt
```

Teraz zobacz nagłówki, żeby poznać ID sesji i słowo:

```
cat headers.txt | grep -E 'Session|Word'
```

♦ W nagłówkach znajdziesz:

- **X-Session-Id:** → identyfikator sesji
- **X-Word:** → słowo do podpisania

♦ A w pliku **response.pem** masz:

- Twój klucz prywatny RSA w formacie PEM (-----BEGIN PRIVATE KEY----- ...)

```
[user@parrot]~  
$ curl -s -D headers.txt -o privkey.pem -X GET 'http://localhost:3009/sign'
```

```
[user@parrot]~  
$ cat headers.txt  
HTTP/1.1 200 OK  
Server: Werkzeug/3.1.3 Python/3.10.19  
Date: Wed, 12 Nov 2025 09:44:55 GMT  
Content-Disposition: attachment; filename=private_key.pem  
X-Session-Id: HEFj_4HlDOzhtd9qCzC7aw  
X-Word: yankee03  
Content-Type: application/x-pem-file  
Content-Length: 1678  
Access-Control-Allow-Origin: *  
Connection: close
```



```

[user@parrot]~$ cat privkey.pem
-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEAncUFhEiD5pjZw1Av8UnqcMJnu7E9GSCIYQYkp80JcoNte0tQ
MWopG/7cYH7SFPuYrYrqeTa7RWzt40qW3bawEDY94RYCQW8WZtmTSjw7Qf0doDAV
X+s0FHTVcRfevE9GxC5fHo05Q5KIaXY/nBeuCaSwRHtJ/AHnsm0RvigiMdf0x7dX
yC+WQCxVyft634N47ejnm1HVExPZvvcYsAJGYA0+f06ZSa8ReLMFzF54oALVLlyz
0kOn5KieLsBIInbEhUzH0FdvUUrIFbFc0bnziniw2I/T9qywjbUyAMEudk4YG1Lyc
paH0e4G9/R3LSOUPfDMaLyOPjOJ6FUWxj2+/BQIDAQABAoIBAAhECTszTRJ54R+E
wLzwnV4NgRpNxFCf8005XBZyMq5uI4A8PI4Y4gZTtro+grqzM3yqJHPiFHLGzNAQ
c1mI3qtmxXIDbBhUzTYbYlW462Q+inq7apswU8uD4ML7Vvqf7URDzjjohj/y8en
Y0NGEVSeLSxVVPB1WCfcDjErMJ7G/D4NHKXYEJ+NcjTk+pFUVdGdAD/2mh3G062M
uZKGWwZT/v+PxQE27q/b7z0IzYgBTBV0Q5Aa6d8qZInBYhg8oL49iQU8jY/RE4uA
yFRcX3cKHe7X1inWFBWqxYy2xg+ThOKJ6JVzU0pHdmX8EDN2U2d1KIEujWerR6Gm
jEs43XECgYEAvmIuPOff/rS1BK24BD21lm6o9kMkwPK9EoDhzgEAAFIhhhd9Mu
0IiseZQVx/XzmyvK1Jnqky950VWdW/t+osmaHwS457/bwhdGOBKqrTBvaQH1yWQ
uLmAALcHCgyTmg/M+eNfBoDRkMtmDsPFfTeD2nmXVgaDIIm37NWM05UCgYEA1e3t
Fu8apYyw4G1C53H03qEJt9d0ugFTCdZkeHw2aFozw90NzZFrXrpvNbRHfgPjsn/j
9eVgNtKt/Z/5RcvIe94mFN2VdoJ3gS3d/9voTrVGfHqmY3dUdMyPL12G51J7x0xX
wK3tAmy3vksp065CLqt0/b8D0KFt6NzTsGbFGbECgYEAqX350bEuZNr10eIjM72H
HKCponc1hr5CRYAcia76qOKbuSRSPJTnLmfFD9EWTNjN76jrZ0NPtd/3jSz+1afK
6mckMDtTae5Q04oyqiaSYfatCGe+cjRTXMOMWxf1xdvMlz8gMiciJGB5w5gXI6I2
Jlg0k2sdpilqh0ecS47bupECgYBkaI/n08KeATLjo2TLjLcz9xHAn/fBde2rqFzv
0B0ZTX00ZLUhzydu1wJrinojw1+T4EL2nvpRwQC5EeTFxagLu3DUSkxkm0fL8eNI
u8aFar/ynP6Lo06bA4gQkwVGBWDiRKvLW9bqVGzJfC3d10SrUux4XLdFrP8RVNRm
71WDgQKBgQCC7N34KoQZ+ha0raQP+vvf2Q6SUYAS6mcB1oC6TQEGrYD/1xPudJWl
hzgYnz5rV9pVZMuUU2aYt80BKHGzeGMLgz5ZzAQpme3SAypBma8T8qZpXKh8Lqw3
WsHCR11uo7500TR3bpB/FbdMu+W52D20sQDmL5LHZI+1EztER8uDgw==
-----END RSA PRIVATE KEY-----
[user@parrot]~$

```

KROK 3 — przygotowanie plików

Zapisz słowo do pliku:

```
echo -n "TUTAJ_WSTAW_SŁOWO" > word.txt
```

SHELL

```

$ echo -n "yankee03" > word.txt

```

Sprawdź:

```
cat word.txt
```

SHELL

KROK 4 — utworzenie skrótu SHA-256

Tworzymy skrót SHA-256 (binarne dane):

```
openssl dgst -sha256 -binary -out word.sha256 word.txt
```

SHELL

```
[user@parrot]-[~]  
$openssl dgst -sha256 -binary -out word.sha256 word.txt
```

Sprawdź długość (powinno być 32 bajty):

```
ls -l word.sha256
```

SHELL

```
[user@parrot]-[~]  
$ls -l word.sha256  
-rw-r--r-- 1 user user 32 Nov 12 09:50 word.sha256
```

KROK 5 — podpisanie słowa RSA-PSS

Użyj klucza prywatnego do podpisania skrótu:

```
openssl pkeyutl -sign -in word.sha256 -inkey response.pem \  
-pkeyopt digest:sha256 \  
-pkeyopt rsa_padding_mode:pss \  
-pkeyopt rsa_pss_saltlen:32 \  
-out signature.bin
```

SHELL

```
[user@parrot]-[~]  
$openssl pkeyutl -sign -in word.sha256 -inkey privkey.pem \  
-pkeyopt digest:sha256 \  
-pkeyopt rsa_padding_mode:pss \  
-pkeyopt rsa_pss_saltlen:32 \  
-out word.bin
```

Teraz zakoduj podpis do base64 (żeby można go było wysłać HTTP-em):

```
base64 signature.bin > signature.b64
```

SHELL

```
[user@parrot]-[~]  
$base64 word.bin > word.b64
```

Sprawdź zawartość:

```
cat signature.b64
```

SHELL

```
[user@parrot]-[~]  
$cat word.b64  
CyWg6sQgD0zCPhjGGnHzG7uj7wDbdzvcjs/Ii8Hb3FIiaJbMa6UExjEUM0yp6dBNedGi1tbczmpu  
DpEtvsQ7ttuMRq58a+vVYKgnCamtT4ecq5ysK4aIAZA3Y4NIrBQ3bhRiMwPvXeUyno4hFFtsm/ws  
lEPrtlkboFYqT/cpShnr/VKTyaIB7TE7f0BfLcAnlCPF08UbtXv+ZvRLt6HNDokQ03YPXi6mHkk  
TDmeJlqh0H+cYCYypUyIOk3Hr0xkkHGw2idUf2riSMbGYWkgmWob5dGQDLq/m2l4YFw+Qe5ztZhd  
x5N5+0ZqWwK+VBDaCoDh+b0mpyHv6UTi+ce8jA==
```



KROK 6 — wysłanie podpisu do serwera

Użyj `curl` do przesłania podpisu (jako base64) i ID sesji:

```
curl -X POST 'http://127.0.0.1:3009/submit' \  
-H 'accept: application/json' \  
-H 'Content-Type: application/json' \  
-d '{  
    "session_id": "TU_WSTAW_SESSION_ID",  
    "signature_b64": "TU_WKLEJ_ZAWARTOŚĆ_signature.b64"  
}'
```

SHELL

```
[user@parrot]-[~]  
$curl -X 'POST' \  
'http://localhost:3009/submit' \  
-H 'accept: application/json' \  
-H 'Content-Type: multipart/form-data' \  
-F 'session_id=HEFj_4HlD0zhtd9qCzC7aw' \  
-F 'signature_b64=@word.b64'  
{"result": "Correct signature!"}
```

3.10 Pod adresem <http://127.0.0.1:3010> działa prosty serwer HTTP udostępniający dwa endpointy: <http://127.0.0.1:3010/verify> oraz <http://127.0.0.1:3010/submit>.

(a) Uruchom serwer za pomocą poniższego polecenia:

```
docker run -p 3010:3010 --name ex10 docker.io/mazurkatarzyna/asymmetric-enc-ex10:latest
podman run -p 3010:3010 --name ex10 docker.io/mazurkatarzyna/asymmetric-enc-ex10:latest
```

Jeśli Docker Hub nie odpowiada, użyj obrazu zapasowego:

```
docker run -p 3010:3010 --name ex10 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex10:latest
podman run -p 3010:3010 --name ex10 ghcr.io/mazurkatarzynaumcs/asymmetric-enc-ex10:latest
```

- (b) Wyślij request do endpointa <http://127.0.0.1:3010/verify> używając metody HTTP GET. Otrzymasz w odpowiedzi klucz publiczny RSA (`public_key_pem`), podpis (`signature_b64`) oraz słowo (`word`).
- (c) Zweryfikuj podpis przy użyciu algorytmu RSA-2048 oraz trybu paddingu PSS. Przy weryfikacji należy zwrócić uwagę na parametry paddingu PSS:
- Używana funkcja skrótu: SHA-256
 - Salt length: długość soli powinna odpowiadać długości skrótu (32 bajty dla SHA-256), aby weryfikacja lokalna była zgodna z serwerem.
- (d) Używając metody HTTP POST, wyślij do serwera poprzez endpoint <http://127.0.0.1:3010/submit> słowo (jako `word`), klucz publiczny (jako `public_key_pem`), podpis w formacie base64 (jako `signature_b64`) oraz informację, czy weryfikacja lokalna zakończyła się sukcesem (`true/false`) (jako `user_verified`).
- (e) Serwer sprawdzi, czy Twoja lokalna weryfikacja jest zgodna z faktycznym podpisem i zwróci odpowiednią informację.

```

[user@parrot]-[~]
└─$ curl -s -D data.txt -o files.zip -X GET http://localhost:3010/verify
[user@parrot]-[~]
└─$ cat data.txt
HTTP/1.1 200 OK
Server: Werkzeug/3.1.3 Python/3.10.19
Date: Fri, 21 Nov 2025 17:55:52 GMT
Content-Disposition: attachment; filename=challenge.zip
X-Session-Id: nhJwDumhk_hntTL_CdvA7g
X-Word: strawberry
Content-Type: application/zip
Content-Length: 881
Access-Control-Allow-Origin: *
Connection: close

[user@parrot]-[~]
└─$ unzip files.zip
Archive:  files.zip
  inflating: public_key.pem
  inflating: signature.b64
[user@parrot]-[~]
└─$ echo -n "strawberry" > word.txt
[user@parrot]-[~]
└─$ base64 -d signature.b64 > signature.bin

```

Żeby przechwycić wynik polecenia do weryfikacji trzeba zrobić zmienną z wynikiem w formie 'true' lub 'false'

```

[user@parrot]-[~]
└─$ result=$(openssl dgst -sha256 -verify public_key.pem -sigopt rsa_padding_mod
e:pss -sigopt rsa_pss_saltlen:32 -signature signature.bin word.txt | grep -q 'Veri
fied OK' && echo true || echo false)
[user@parrot]-[~]
└─$ echo $result
true

```

Należy pamiętać o "" przy zmiennej w user_verified bo gdy podajemy zmienną w tym zadaniu musi być przekazany string a nie wartość true/false !!!

```
[user@parrot]~$ curl -X 'POST' \
  'http://localhost:3010/submit' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'session_id=nhJwDumhk_hntTL_CdvA7g' \
  -F 'word=strawberry' \
  -F 'public_key_pem=@public_key.pem' \
  -F 'signature_b64=@signature.b64' \
  -F "user_verified=$result"
{"result": "Correct! Your verification matches the actual signature."}
```